

Interactive Mandelbrot Explorer: Datastar + FastHTML + Mojo GPU

Overview

This is a reactive web app for exploring the Mandelbrot set. The frontend uses Datastar for signals and real-time updates (e.g., sliders for zoom/position). The backend is FastHTML (Python) handling routing and HTML generation. Compute-intensive rendering uses a Mojo GPU kernel for fast iteration counts on the GPU. Python calls Mojo, generates a color-mapped PNG, and returns base64-encoded image fragments for Datastar to merge.

Key features:

- Interactive sliders update view bounds via signals.
- GPU-accelerated computation for near-real-time rendering.
- Server-side partial updates for smooth UX.

Requirements

- **Python:** 3.10+ with `pip install python-fasthtml ft-datastar pillow`
- **Mojo:** MAX SDK installed (e.g., via Modular tools: `magic install max`). GPU required (NVIDIA/AMD/Intel with drivers).
- **Datastar JS:** Loaded via CDN (v1.0.0-beta.8 or latest).
- **Browser:** Modern (supports ES modules for Datastar).

Project Structure

```
text
```

```
project/
```

```
|— main.py           # FastHTML app: routes, Datastar integration, Mojo calls  
└— mandelbrot.mojo  # Mojo GPU kernel: computes iteration tensor
```

Code: mandelbrot.mojo

```
mojo

from python import PyObject
from tensor import Tensor, TensorShape
from math import Complex
from gpu import cluster, id, intrinsics # GPU intrinsics from MAX
from algorithm import parallelize

alias MAX_ITERS = 256
alias DTYPE = DType.float32

@kernel # Marks as GPU kernel
fn mandelbrot_kernel(
    output: Tensor[Int32],
    width: Int,
    height: Int,
    xmin: Float32,
    xmax: Float32,
    ymin: Float32,
    ymax: Float32
) -> None:
    let gx = id.global_x() # Thread global ID x
    let gy = id.global_y() # Thread global ID y
    let stride_x = cluster.grid_dim_x() * cluster.block_dim_x()
    let stride_y = cluster.grid_dim_y() * cluster.block_dim_y()

    for x in range(gx, width, stride_x):
        for y in range(gy, height, stride_y):
            let cx = xmin + (xmax - xmin) * x.to_float32() / (width - 1).to_f
            let cy = ymin + (ymax - ymin) * y.to_float32() / (height - 1).to_f
            var z = Complex[DTYPE](0.0, 0.0)
            var c = Complex[DTYPE](cx, cy)
            var iters = 0
            while z.magnitude_squared() <= 4.0 and iters < MAX_ITERS:
                z = z * z + c
                iters += 1
            output[Index(x, y)] = iters

fn compute_mandelbrot(
    width: Int,
    height: Int,
```

```

    xmin: Float32,
    xmax: Float32,
    ymin: Float32,
    ymax: Float32
) raises -> Tensor[Int32]:
    let shape = TensorShape(width, height)
    var output = Tensor[Int32](shape)

    # Dispatch kernel on GPU: Define grid/block sizes
    let block_size = intrinsics.Dim(16, 16) # 16x16 threads per block
    let grid_size = intrinsics.Dim(
        (width + block_size.x - 1) // block_size.x,
        (height + block_size.y - 1) // block_size.y
    )
    intrinsics.dispatch[mandelbrot_kernel](
        grid_size, block_size, output, width, height, xmin, xmax, ymin, ymax
    )

    # Sync GPU
    intrinsics.synchronize()
    return output

# Export for Python interop
@export
fn PyInit_mandelbrot() -> PythonObject:
    var m = PythonModuleBuilder("mandelbrot")
    m.def_function[compute_mandelbrot]("compute_mandelbrot")
    return m.finalize()

```

Code: main.py

```

Python

from fasthtml.common import *
from ft_datastar import * # Datastar helpers
from PIL import Image # For image generation
import base64
from io import BytesIO
import mojo.importer # Enable Mojo imports
import mandelbrot # Imports/Compiles mandelbrot.mojo

# Datastar JS CDN

```

```

datastar_script = Script(
    src="https://cdn.jsdelivr.net/npm/@sudodevnull/datastar@v1.0.0-beta.8/bun
    type="module"
)

app, rt = fast_app(hdrs=(datastar_script,))

def color_map(iters, max_iters=256):
    """Simple color gradient: black to white via hues."""
    if iters == max_iters: return (0, 0, 0)
    hue = int(255 * (iters / max_iters))
    return (hue, 255 - hue % 128, 255 - hue)

@rt("/")
def get():
    return Div(
        H2("Interactive Mandelbrot with Mojo GPU", cls="text-xl font-bold mb-
        # Signals for view bounds (initial: full set)
        Div(ds_signals(xmin="-2.0", xmax="1.0", ymin="-1.5", ymax="1.5", zoom
        # Sliders for interaction
        Label("Zoom: ", Input(type="range", min="1", max="100", value="1", ds
        Label("X Center: ", Input(type="range", min="-2", max="1", step="0.1"
        Label("Y Center: ", Input(type="range", min="-1.5", max="1.5", step="
        # Image placeholder
        Img(id="mandel-img", src="", alt="Mandelbrot", cls="w-full h-auto bor
        # Trigger recompute on signal changes
        Div(ds_on(signal_changed="$xmin, $xmax, $ymin, $ymax, $zoom", "GET /r
        cls="p-4 space-y-4"
    )

@rt("/render")
def get(request):
    # Parse signals from query (Datastar sends them)
    xmin = float(request.query.get("xmin", -2.0))
    xmax = float(request.query.get("xmax", 1.0))
    ymin = float(request.query.get("ymin", -1.5))
    ymax = float(request.query.get("ymax", 1.5))
    zoom = float(request.query.get("zoom", 1.0))

    # Adjust bounds based on zoom/center (derived signals)
    xcenter = (xmin + xmax) / 2
    ycenter = (ymin + ymax) / 2
    width = (xmax - xmin) / zoom

```

```

height = (ymax - ymin) / zoom
xmin, xmax = xcenter - width / 2, xcenter + width / 2
ymin, ymax = ycenter - height / 2, ycenter + height / 2

# Call Mojo GPU kernel
WIDTH, HEIGHT = 800, 600 # Resolution (tune for perf)
iters_tensor = mandelbrot.compute_mandelbrot(WIDTH, HEIGHT, xmin, xmax, y

# Convert tensor to image (assume [] access on Tensor)
img = Image.new("RGB", (WIDTH, HEIGHT))
pixels = img.load()
for x in range(WIDTH):
    for y in range(HEIGHT):
        iters = iters_tensor[x, y]
        pixels[x, y] = color_map(iters)

# Base64 encode
buffered = BytesIO()
img.save(buffered, format="PNG")
img_b64 = base64.b64encode(buffered.getvalue()).decode()

# Return fragment for Datastar merge
return Img(src=f"data:image/png;base64,{img_b64}", id="mandel-img", ds_me

serve()

```

How to Run

1. Place files in a directory.
2. Run `python main.py` (starts Uvicorn server at `http://localhost:8000`).
3. Open in browser: Interact with sliders for real-time GPU-rendered updates.

Notes

- **Mojo Interop:** Compiles on first import; tune grid/block sizes for your GPU.
- **Perf:** 800x600 resolution is balanced; increase for detail (GPU handles it fast).
- **Extensions:** Add SSE for progressive rendering or more controls (e.g., max_iters slider).
- **Limitations:** Assumes Mojo Tensor has NumPy-like access; adjust if API evolves. No error handling shown—add for production.